

苏剑林研究专题 10: MHA -> MQA -> GQA -> MLA

KV-cache、低秩压缩与现代推理架构

2026-08-14

Contents

摘要	1
1. MHA: 每个 Query head 都有自己的 K/V	2
2. KV-cache 的量级	2
3. MQA: 所有 Q head 共用一组 K/V	2
4. GQA: 在质量与 cache 间折中	2
5. 为什么 decode 阶段特别受益	2
6. MLA: 把 K/V 压到 latent space	3
7. Decoupled RoPE: 为什么位置部分要特殊处理	3
8. Latent compression 与低秩假设	3
9. 参数 FLOPs 与 cache 的 trade-off	3
10. MHA/MQA/GQA/MLA 对比	3
11. Gotchas	4
11.1 只按 KV 元素数估吞吐	4
11.2 MLA 训练实现和推理解码实现可能不同	4
11.3 RoPE dimension 是关键超参数	4
12. Know-how: 如何做系统评测	4
13. Know-why	4
复习清单	4
参考资料 (精选)	4

系列定位。本报告属于“苏剑林研究专题”系列。报告将三类内容明确区分: (1) 苏剑林本人署名论文/项目; (2) 其在“科学空间”中长期公开推导与分析的研究性文章; (3) 其参与的团队论文 (例如 Kimi 系列)。对于团队工作, 不把公开论文无法确认的模块主导权归因给某一位作者。

阅读方法。不建议只记结论。每一节都尽量按“问题 -> 数学约束 -> 结构 -> 工程实现 -> 边界条件”展开, 并单独列出 gotchas、know-how 与 know-why。

摘要

MHA -> MQA -> GQA -> MLA 的演化, 本质上是在优化一个越来越关键的系统变量: **KV-cache**。随着模型上下文变长、batch 增大, 解码时参数 FLOPs 往往不再是唯一瓶颈, K/V 的显存与内存带宽成为决定吞吐的核心。苏剑林对这条 Attention 演化路线做过系统分析。本专题从 MHA 的张量形状出发, 定量解释每一步压缩了什么、损失了什么, 以及 MLA 为什么不是“另一种少 KV head”。

1. MHA: 每个 Query head 都有自己的 K/V

设模型 hidden size d , head 数 H , 每头维度 $d_h = d/H$ 。MHA 计算

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

拆成 head 后:

$$Q, K, V \in \mathbb{R}^{B \times H \times L \times d_h}.$$

自回归解码时过去 token 的 K/V 可缓存, 不必重复计算。但每新增一个 token, 都要保存每层每 head 的 K/V。

2. KV-cache 的量级

每层每 token 的 KV 元素数近似

$$2Hd_h = 2d.$$

总 cache (忽略对齐) 近似

$$M_{KV} \approx B \cdot L \cdot N_{layer} \cdot 2d \cdot \text{bytes}.$$

例如 context 增长 32 倍, KV-cache 也近似增长 32 倍。这就是长上下文推理的第一性成本。

3. MQA: 所有 Q head 共用一组 K/V

Multi-Query Attention 保留多个 Query head, 但 K/V 只有一个 head:

$$K, V \in \mathbb{R}^{B \times 1 \times L \times d_h}.$$

KV-cache 理论上相对 MHA 减少约 H 倍。

代价是所有 query head 共享同一 key/value 子空间, 表示能力可能下降。

4. GQA: 在质量与 cache 间折中

Grouped-Query Attention 设 KV head 数为 G , 且 $1 < G < H$ 。每 H/G 个 Q head 共享一个 KV head。cache 相对 MHA 减少约

$$\frac{H}{G}$$

倍, 同时保留比 MQA 更丰富的 K/V 表示。现代开源 LLM 中 GQA 非常常见。

5. 为什么 decode 阶段特别受益

Prefill 阶段需要大矩阵 Attention, 通常 compute-heavy; decode 阶段每次只有一个或少量 query, 却要读取整个历史 K/V, 容易 memory-bandwidth-bound。

因此减少 KV-cache 不仅省显存, 也减少每步从 HBM 读取的数据量, 可能直接提高 tokens/s。

6. MLA: 把 K/V 压到 latent space

Multi-head Latent Attention 的思想是先把 hidden state 压缩到较低维 latent:

$$c_t = W_D x_t,$$

需要使用 K/V 时再通过上投影恢复或等价重参数化:

$$k_t = W_K^U c_t, \quad v_t = W_V^U c_t.$$

cache 的主要对象变成较小的 c_t , 而不是完整 per-head K/V。

这与 GQA 的差别是: GQA 减少 KV head 数; MLA 通过低秩/latent 表示压缩跨 head 的 K/V 信息。

7. Decoupled RoPE: 为什么位置部分要特殊处理

如果 K 被完全压到 latent 后再与不同 head 的 Q 交互, RoPE 的位置旋转与低秩吸收会出现代数兼容问题。MLA 常把与 RoPE 相关的 key 分量单独处理, 使其不能被简单吸收入缓存低秩路径的部分保持显式。

这说明高效 Attention 设计需要同时考虑:

- 表示压缩;
- 位置编码;
- 推理重参数化。

8. Latent compression 与低秩假设

MLA 隐含假设是: 完整 K/V 在 head 维上的信息存在较低维结构, 可以通过 latent 保存主要信息。

如果真实最优 K/V 完全高秩, 那么强压缩会损失能力。实际成功说明至少在大模型训练中, 网络可以适应这种瓶颈并把有用信息组织到 latent 中。

9. 参数 FLOPs 与 cache 的 trade-off

MLA 为压缩/解压增加额外 projection, 但推理中 cache 读写减少。是否更快取决于硬件:

$$\text{extra GEMM cost} \quad \text{vs} \quad \text{saved memory bandwidth}.$$

这也是为什么理论参数量或 FLOPs 不能单独判断 serving 性能。

10. MHA/MQA/GQA/MLA 对比

结构	KV 表示	cache	主要优点	主要风险
MHA	每 Q head 独立 KV	最大	表达自由	长上下文 cache 大
MQA GQA	单组 KV 分组 KV	最小之一 中等	极省 cache 成熟折中	表示瓶颈明显 仍随 KV heads 增长
MLA	latent + 位置分量	很小	低秩压缩、解码高效	实现/重参数化复杂

11. Gotchas

11.1 只按 KV 元素数估吞吐

cache 更小不保证 end-to-end 更快；kernel fusion、quantization、batch shape 和 tensor parallel 通信都影响实际速度。

11.2 MLA 训练实现和推理解码实现可能不同

很多投影在推理时可以代数吸收/合并。如果直接按训练图 serving，可能失去 MLA 的主要收益。

11.3 RoPE dimension 是关键超参数

只对一部分维度使用 RoPE 时，这部分既影响位置能力也影响不能被压缩的 cache 大小。

12. Know-how：如何做系统评测

同时测：

- prefill TFLOPs/s;
- decode tokens/s;
- HBM read bandwidth;
- 每 token 每层 KV bytes;
- batch/context 扩展曲线;
- perplexity / downstream;
- 长上下文 retrieval。

只比较单 batch、短 context 的速度很容易得出错误结论。

13. Know-why

这条演化链说明现代 Attention 研究已经从“如何算更好的相关性”扩展为“如何设计一个能被硬件高效缓存和读取的表示”。推理状态本身成为模型架构的一部分。

复习清单

建议在完成本专题后，能够回答下面四类问题：

1. **定义题**：核心对象、公式和变量分别是什么？
2. **推导题**：为什么这种结构能够解决原问题？关键等式如何得到？
3. **工程题**：实现时真正容易出错的地方是什么？哪些超参数决定行为？
4. **迁移题**：如果数据规模、上下文长度、模型宽度或训练预算变化，原结论还成立吗？

参考资料（精选）

- 科学空间：MHA、MQA、GQA、MLA 系列分析—<https://spaces.ac.cn/archives/10091>
- DeepSeek-V2（MLA 公开来源之一）—<https://arxiv.org/abs/2405.04434>
- RoFormer —<https://arxiv.org/abs/2104.09864>

版本说明：2026-08-14。本文以公开论文、公开项目与“科学空间”公开文章所形成的研究脉络为基础，重点用于技术学习与研究复盘，而非作者个人传记。